

Plan de Desarrollo y Pruebas

PyME Financing Platform - Fases, criterios de salida y validacion

1. Objetivo

Definir el orden de construccion del sistema y las pruebas minimas necesarias para considerar estable cada modulo. Este plan evita desarrollar funcionalidades fuera de orden y reduce retrabajo.

Criterio general

Cada fase debe terminar con commit, pruebas basicas y verificacion manual. No avanzar a la siguiente fase si la anterior rompe autenticacion, permisos, datos o migraciones.

2. Plan de desarrollo por fases

Fase	Objetivo	Entregables	Criterio de salida
0	Preparacion del repositorio	Monorepo, .env.example, README inicial, Docker Compose base.	docker compose levanta servicios base sin errores.
1	Base de datos y Prisma	Schema Prisma, migraciones, seeders de roles, documentos, productos y reglas.	Migraciones aplican en base limpia y seed carga datos iniciales.
2	Autenticacion y roles	Login, JWT, users, roles, guards basicos.	internal_operator y applicant pueden iniciar sesion y ver solo rutas permitidas.
3	Empresas	CRUD de companies con propiedad applicant_user_id y created_by_user_id.	Applicant solo lista sus empresas; operador lista todas.
4	Solicitudes	CRUD de financing_applications, datos financieros y status inicial.	Solicitud se crea asociada a empresa valida y respeta permisos.
5	Documentos	Checklist, carga local, metadatos, revision y descarga segura.	Archivo se guarda localmente y BD conserva ruta/metadatos.
6	Productos y reglas	financial_products y product_rules con is_active.	Operador crea/edita productos y reglas; applicant no accede.
7	Analisis de riesgo	Calculo de riesgo, input_snapshot, historial y razones.	No se calcula si faltan precondiciones. Cada recalculation crea registro nuevo.
8	Matching	application_matches por risk_assessment_id.	Cada analisis genera recomendaciones propias sin duplicar producto en el mismo analisis.
9	Expediente y dashboard	Vista consolidada de empresa, solicitud, documentos, riesgo, matches e historial.	El expediente muestra ultimo analisis y matches correctos por rol.
10	Auditoria, pruebas y pulido	audit_logs, pruebas, README final, capturas y datos demo.	Flujos principales funcionan desde clonacion del repo.

3. Estrategia de pruebas

Tipo	Objetivo	Herramienta sugerida
Unitarias	Validar funciones puras: calculo de riesgo, DSCR, evaluador de reglas.	Jest
Integracion	Probar servicios con base de datos de test.	Jest + Prisma + Postgres test
API	Probar endpoints, permisos y errores.	Supertest / Postman
E2E manual	Validar flujo completo desde frontend.	Navegador
Seguridad basica	Probar acceso cruzado entre applicants.	Tests API
Archivos	Validar carga, rechazo, descarga y permisos.	Tests API + filesystem

4. Pruebas minimas por modulo

Modulo	Pruebas minimas
auth	Login correcto, login fallido, usuario inactivo, token faltante, token invalido.

Modulo	Pruebas minimas
companies	Crear empresa, RFC duplicado, applicant ve solo propias, operador ve todas.
applications	Crear solicitud, actualizar datos financieros, bloquear company_id no permitido.
documents	Inicializar checklist, subir PDF valido, rechazar extension invalida, aprobar/rechazar solo operador.
products	Crear producto, editar producto, desactivar producto, bloquear applicant.
rules	Crear regla valida, rechazar operator/value incoherente, desactivar regla.
risk	No calcular con has_existing_debt NULL, calcular con datos completos, guardar input_snapshot.
matching	Generar matches por risk_assessment_id, evitar duplicado por producto, calcular score 0..100.
status	Permitir transicion valida, rechazar draft -> matched, registrar historial.
audit	Crear audit_log en acciones sensibles, no guardar password_hash ni tokens.

5. Flujos de prueba integrales

Flujo	Pasos	Resultado esperado
Flujo interno completo	Operador crea empresa -> solicitud -> documentos -> revision -> riesgo -> matching -> expediente.	Caso llega a matched con analisis y recomendaciones visibles.
Flujo applicant	Applicant crea empresa -> solicitud -> sube documentos -> consulta estado.	No puede revisar documentos ni calcular riesgo.
Acceso cruzado	Applicant A intenta ver solicitud de Applicant B.	API responde 403 o 404 controlado.
Recalculo de riesgo	Modificar datos -> generar nuevo risk_assessment -> generar matches.	No se sobrescribe historial anterior.
Producto desactivado	Desactivar producto -> generar matching nuevo.	Producto inactivo no se recomienda en nuevos matches.

6. Definition of Done por fase

Criterio	Descripcion
Compila	Frontend y backend inician sin errores.
Migraciones	La base se puede crear desde cero con migraciones y seed.
Permisos	Las rutas criticas validan rol y propiedad en backend.
Errores	Errores devuelven formato estandar y mensajes claros.
Auditoria	Acciones sensibles generan audit_logs cuando aplique.
Pruebas	Pruebas minimas de la fase pasan.
Documentacion	README o docs indican como probar la fase.
Commit	Cada fase termina con commit claro y descriptivo.

7. Datos demo y seeders

Seeder	Contenido minimo
roles	internal_operator, applicant.
users	Un operador demo y un applicant demo.
document_requirements	Identificacion, RFC, comprobante domicilio, estados de cuenta, acta, poder, autorizacion buro, facturas, cotizacion.
financial_products	Factoraje, credito simple, capital de trabajo, linea revolving, arrendamiento.
product_rules	Reglas basicas por need_type, facturas, urgencia, antiguedad, riesgo y DSCR.
demo case	Empresa, solicitud y documentos simulados para mostrar el flujo rapidamente.

8. Riesgos del desarrollo

Riesgo	Consecuencia	Mitigacion
Sobre-diseñar	El proyecto se vuelve demasiado grande.	Respetar MVP y fases.
Permisos solo en frontend	Usuarios acceden a datos ajenos por API.	Guards y ownership en backend.

Riesgo	Consecuencia	Mitigacion
Archivos inseguros	Subida de ejecutables o descarga no autorizada.	Validar tipo, tamaño, ruta y permisos.
Calculos con float	Errores de precision financiera.	Usar Decimal y redondeo controlado.
Reglas incoherentes	Scores fuera de rango o recomendaciones malas.	Limitar score_weight y normalizar score 0..100.
No guardar snapshot	Historial de riesgo pierde contexto.	input_snapshot obligatorio.

9. Orden de commits recomendado

```

feat: initialize monorepo and docker compose
feat: add prisma schema and database migrations
feat: seed roles document requirements and financial products
feat: implement auth and role guards
feat: implement companies module
feat: implement financing applications module
feat: implement local document upload and review
feat: implement financial products and product rules
feat: implement risk assessment calculation
feat: implement product matching by risk assessment
feat: implement credit file dashboard
feat: add audit logs and final tests

```

Anexo de Actualización

PyME Financing Platform

Documento base conservado: Plan de Desarrollo y Pruebas

Este anexo no reemplaza ni resume el documento base. El contenido original se conserva íntegro. Las reglas de este anexo se agregan al diseño y prevalecen cuando una regla anterior sea ambigua o use lenguaje de sugerencia.

A. Decisiones confirmadas para todos los documentos

Decisión	Instrucción obligatoria
Registro de applicant	El applicant debe poder crear su propia cuenta desde una ruta pública usando, como mínimo, fullName, email y password. El sistema debe asignar el rol applicant automáticamente.
Resultado visible para applicant	El applicant no debe ver el análisis interno completo, risk_score, pesos de reglas, DSCR detallado, audit_logs ni comparación interna de productos. Solo debe ver la decisión final publicada por el operador.
Operación interna	El internal_operator debe ver risk_assessments, application_matches, razones internas, porcentajes, indicadores financieros y debe crear/publicar la decisión final.
Corrección de necesidad	El internal_operator debe poder validar o corregir la necesidad declarada por el applicant mediante validated_need_type antes de generar matching o decisión final.
Documentos rechazados	Cuando el applicant suba nuevamente un documento rechazado, el sistema debe reemplazar el archivo en el mismo application_document, actualizar metadatos, cambiar status a uploaded y registrar audit_log.
Auditoría	El sistema debe auditar solo acciones sensibles; no debe registrar absolutamente todos los cambios ni guardar secretos, tokens, password_hash o contenido de documentos.

B. Regla de redacción para implementación

Toda frase del documento base que use “recomendado”, “sugerido”, “podría” o “si se implementa” debe interpretarse como instrucción obligatoria cuando describa una regla del MVP, una validación de seguridad, una entidad de base de datos, un endpoint crítico, una transición de estado o una prueba mínima.

C. Plan ajustado sin eliminar fases anteriores

Fase	Agregar a los entregables existentes
2 Auth y roles	Agregar POST /auth/register para applicant y pruebas de email duplicado/password inválido.
4 Solicitudes	Agregar validated_need_type, validación del operador y auditoría validate_need_type.
5 Documentos	Agregar prueba de reemplazo de documento rechazado en el mismo application_document.
7 Análisis de riesgo	Confirmar que applicant no puede consultar risk_assessments.
8 Matching	Usar validated_need_type si existe y probar que productos incoherentes sean penalizados o descartados.
9 Expediente y dashboard	Agregar vista interna de decisión para operador y vista pública limitada para applicant.
10 Auditoría, pruebas y pulido	Agregar pruebas de application_decisions, publicación y acceso applicant.

Pruebas mínimas agregadas

- Applicant se registra, inicia sesión y no puede elegir rol.
- Operador corrige need_type y el matching usa validated_need_type.

- Applicant no ve risk_score ni application_matches internas.
- Operador crea y publica application_decision.
- Applicant ve la decisión solo después de publicada.
- Documento rechazado se reemplaza en el mismo registro y genera audit_log.